# Sorting Algorithms in Python

This repository contains Python implementations of five fundamental sorting algorithms. Each program takes user input, sorts the array, displays the sorted output, prints the time complexity, and measures the execution time in microseconds.

## 📁 Files Included

- `bubblesort.py` – Bubble Sort
- `insertionsort.py` – Insertion Sort
- `selectionsort.py` – Selection Sort
- `mergesort.py` – Merge Sort
- `quicksort.py` – Quick Sort

## 🚀 Features

- User input for array elements
- Displays sorted array
- Measures execution time using `time.perf_counter()`
- Prints Best, Average, and Worst Case Time Complexity
- Easy-to-understand implementation for beginners

## 🛠 Requirements

- Python 3.x

No external libraries are required.

## ▶ How to Run

Clone the repository:

```
git clone https://github.com/your-username/sorting-algorithms-python.git
```

Navigate to the project folder:

```
cd sorting-algorithms-python
```

Run any sorting algorithm:

```
python bubblesort.py
```

or

```
python insertionsort.py
```

or

```
python selectionsort.py
```

or

```
python mergesort.py
```

or

```
python quicksort.py
```

## 📥 Sample Input

```
Enter number of elements:
5

Enter elements:
5 3 1 4 2
```

## 📤 Sample Output

```
Sorted Array:
1 2 3 4 5

Time Complexity:
Best Case : O(n)
Average Case : O(n²)
Worst Case : O(n²)

Execution Time: 25.67 microseconds
```

## 📊 Time Complexity Comparison

| Algorithm | Best Case | Average Case | Worst Case |
|---|---|---|---|
| Bubble Sort | O(n) | O(n²) | O(n²) |
| Insertion Sort | O(n) | O(n²) | O(n²) |
| Selection Sort | O(n²) | O(n²) | O(n²) |
| Merge Sort | O(n log n) | O(n log n) | O(n log n) |
| Quick Sort | O(n log n) | O(n log n) | O(n²) |

## 🖨 Learning Objectives

This project helps in understanding:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort
- Time Complexity Analysis
- Execution Time Measurement in Python

## 🤝 Contributing

Contributions are welcome! Feel free to fork this repository and submit a pull request.

## 📄 License

This project is open-source and available under the MIT License.

---

⭐ If you found this project useful, consider giving it a star on GitHub!

# Algorithms: Factorial, Coin Change & Matrix Chain Multiplication

## 📌 Overview

This project contains implementations of several fundamental algorithms using **Iterative and Recursive methods**.

The project demonstrates how different approaches can be used to solve computational problems and helps in understanding **algorithm design, recursion, iteration, dynamic programming, and time complexity**.

## 📂 Algorithms Included

1. **Factorial**
2. **Coin Change**
3. **Matrix Chain Multiplication**
4. **Iterative Method**
5. **Recursive Method**

---

## 1. 🔢 Factorial

The factorial of a non-negative integer `n` is the product of all positive integers from `1` to `n`.

### Formula

```
n! = n × (n-1) × (n-2) × ... × 1
```

Example:

```
5! = 5 × 4 × 3 × 2 × 1
   = 120
```

### Methods

#### Iterative Factorial

The iterative approach uses a loop to calculate the factorial.

**Time Complexity:** `O(n)` **Space Complexity:** `O(1)`

#### Recursive Factorial

The recursive approach calls the same function repeatedly until it reaches the base case.

```
factorial(n) = n × factorial(n-1)
factorial(0) = 1
```

**Time Complexity:** `O(n)` **Space Complexity:** `O(n)` due to the recursion stack.

---

## 2. 🪙 Coin Change

The Coin Change problem determines the number of ways to make a particular amount using a given set of coins.

### Example

Given:

```
Coins = [1, 2, 5]
Amount = 5
```

Possible combinations include:

```
5
2 + 2 + 1
2 + 1 + 1 + 1
1 + 1 + 1 + 1 + 1
```

The problem can be solved using **Dynamic Programming**.

### Dynamic Programming Approach

A table is maintained where each position represents the number of ways to create a particular amount.

**Time Complexity:** $O(n \times amount)$ **Space Complexity:** $O(amount)$

Where:

- $n$ = number of different coins
- $amount$ = target amount

---

## 3. ⛓ Matrix Chain Multiplication

Matrix Chain Multiplication is an optimization problem used to determine the most efficient way to multiply a sequence of matrices.

The goal is **not to change the order of matrices**, but to find the best placement of parentheses to minimize the number of scalar multiplications.

### Example

For:

```
A × B × C
```

There are two possible arrangements:

```
(A × B) × C
```

or

```
A × (B × C)
```

Both produce the same final matrix, but they may require different numbers of calculations.

### Dynamic Programming

Matrix Chain Multiplication is commonly solved using Dynamic Programming.

**Time Complexity:** $O(n^3)$ **Space Complexity:** $O(n^2)$

---

## 4. 🔁 Iterative Method

An iterative algorithm repeats a set of instructions using loops such as:

```
for
while
```

### Example

```
def factorial(n):
    result = 1

    for i in range(1, n + 1):
        result *= i

    return result
```

### Advantages

- Uses less memory in many cases.
- Generally faster than recursion.
- No recursion stack overhead.
- Suitable for large inputs.

### Disadvantages

- Some problems are harder to express iteratively.
- Code can become more complex for problems with recursive structures.

---

## 5. 🔁 Recursive Method

A recursive algorithm is a function that calls itself to solve smaller versions of the same problem.

### Example

```
def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)
```

### Advantages

- Simple and easy to understand for recursive problems.
- Useful for tree and graph algorithms.
- Often results in shorter code.

### Disadvantages

- Uses additional memory because of the recursion stack.
- Can be slower due to function-call overhead.
- Deep recursion may cause a stack overflow.

---

## 📊 Comparison

---

| Algorithm | Approach | Time Complexity | Space Complexity |
|---|---|---|---|
| Factorial | Iterative | O(n) | O(1) |
| Factorial | Recursive | O(n) | O(n) |
| Coin Change | Dynamic Programming | O(n × amount) | O(amount) |
| Matrix Chain Multiplication | Dynamic Programming | $O(n^3)$ | $O(n^2)$ |

## 🛠 Technologies Used

- **Python**
- Algorithms
- Recursion
- Iteration
- Dynamic Programming

## 🚀 How to Run

### 1. Clone the repository

```
git clone <repository-url>
```

### 2. Open the project directory

```
cd <project-directory>
```

### 3. Run the Python files

For example:

```
python factorial.py
python coin_change.py
python matrix_chain.py
```

## 📂 Suggested Project Structure

```
Algorithms/
│
├── factorial.py
├── coin_change.py
├── matrix_chain.py
├── iterative.py
├── recursive.py
└── README.md
```

## 🎯 Learning Objectives

This project helps in understanding:

- Basic algorithm design
- Iterative programming
- Recursive programming
- Dynamic Programming
- Optimization techniques

- Time complexity
- Space complexity
- Problem-solving techniques

---

## 🔩 Conclusion

The algorithms included in this project demonstrate different approaches to solving computational problems. **Factorial** illustrates iteration and recursion, while **Coin Change** and **Matrix Chain Multiplication** demonstrate the use of Dynamic Programming for optimization and combinatorial problems.

Understanding these algorithms provides a strong foundation for learning **Data Structures and Algorithms (DSA)**.

# Graph Algorithms in Python

This repository contains simple Python implementations of important **Graph Algorithms** with user input.

The project covers:

- **BFS (Breadth-First Search)**
- **DFS (Depth-First Search)**
- **Prim's Algorithm**
- **Kruskal's Algorithm**

These algorithms are useful for understanding graph traversal, connectivity, and Minimum Spanning Trees (MST).

---

## 📌 Algorithms Included

### 1. Breadth-First Search (BFS)

BFS is a graph traversal algorithm that visits vertices **level by level**.

It uses a **Queue** data structure.

**Time Complexity:**

- Best Case: `O(V + E)`
- Average Case: `O(V + E)`
- Worst Case: `O(V + E)`

**Space Complexity:** `O(V)`

Where:

- `V` = Number of vertices
- `E` = Number of edges

---

### 2. Depth-First Search (DFS)

DFS is a graph traversal algorithm that explores as far as possible along one branch before backtracking.

It can be implemented using **recursion or a stack**.

**Time Complexity:**

- Best Case: `O(V + E)`
- Average Case: `O(V + E)`
- Worst Case: `O(V + E)`

**Space Complexity:** `O(V)`

---

### 3. Prim's Algorithm

Prim's Algorithm is a **Greedy Algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

It starts from a selected vertex and repeatedly adds the minimum-weight edge that connects a vertex in the MST to a vertex outside the MST.

**Time Complexity:**

- Using adjacency matrix: `O(V²)`
- Using priority queue: `O(E log V)`

**Space Complexity:** `O(V + E)`

### 4. Kruskal's Algorithm

Kruskal's Algorithm is a **Greedy Algorithm** used to find the **Minimum Spanning Tree (MST)**.

It sorts all edges according to their weights and then adds the smallest edge if it does not create a cycle.

It commonly uses the **Disjoint Set / Union-Find** data structure.

**Time Complexity:**

- Best Case: `O(E log E)`
- Average Case: `O(E log E)`
- Worst Case: `O(E log E)`

**Space Complexity:** `O(V + E)`

---

## 📁 Project Structure

```
Graph-Algorithms/
│
├── BFS.py
├── DFS.py
├── Prims.py
├── Kruskals.py
└── README.md
```

---

## 🛠 Requirements

- Python 3.x
- No external libraries are required.

Check your Python version:

```
python --version
```

---

## ▶ How to Run

Clone the repository:

```
git clone https://github.com/your-username/Graph-Algorithms.git
```

Move into the project folder:

```
cd Graph-Algorithms
```

Run BFS:

```
python BFS.py
```

Run DFS:

```
python DFS.py
```

Run Prim's Algorithm:

```
python Prims.py
```

Run Kruskal's Algorithm:

```
python Kruskals.py
```

## 🖨️ User Input

The programs are designed to accept graph information from the user.

Depending on the algorithm, the input may include:

```
Number of vertices
Number of edges
Edges
Edge weights
Starting vertex
```

Example:

```
Enter number of vertices: 5
Enter number of edges: 6

Enter edge 1: 0 1
Enter edge 2: 0 2
Enter edge 3: 1 3
Enter edge 4: 1 4
Enter edge 5: 2 4
Enter edge 6: 3 4

Enter starting vertex: 0
```

## 📊 Comparison of Algorithms

| Algorithm | Type | Main Purpose | Data Structure | Time Complexity |
|---|---|---|---|---|
| BFS | Traversal | Graph traversal | Queue | `O(V + E)` |
| DFS | Traversal | Graph traversal | Stack/Recursion | `O(V + E)` |
| Prim's | Greedy | Minimum Spanning Tree | Priority Queue/Matrix | `O(E log V)` / `O(V²)` |
| Kruskal's | Greedy | Minimum Spanning Tree | Union-Find | `O(E log E)` |

## 🔍 BFS vs DFS

| Feature | BFS | DFS |
|---|---|---|
| Full Form | Breadth-First Search | Depth-First Search |
| Approach | Level by level | Depth first |
| Data Structure | Queue | Stack/Recursion |
| Shortest path in unweighted graph | Yes | Not guaranteed |
| Time Complexity | `O(V + E)` | `O(V + E)` |
| Space Complexity | `O(V)` | `O(V)` |

## 🌳 Prim's vs Kruskal's

| Feature | Prim's Algorithm | Kruskal's Algorithm |
|---|---|---|
| Type | Greedy | Greedy |
| Purpose | Minimum Spanning Tree | Minimum Spanning Tree |
| Approach | Starts from a vertex | Starts from smallest edge |
| Main Data Structure | Priority Queue | Union-Find |
| Cycle Handling | Naturally avoided | Explicitly checked |
| Suitable for | Dense graphs | Sparse graphs |
| Time Complexity | `O(E log V)` | `O(E log E)` |

## 🎯 Applications

### BFS

- Shortest path in unweighted graphs
- Network broadcasting
- Web crawling
- Social network analysis

### DFS

- Cycle detection
- Topological sorting
- Maze solving
- Connected components

### Prim's Algorithm

- Network design
- Electrical grid design
- Road construction
- Computer network optimization

### Kruskal's Algorithm

- Network design
- Minimum-cost connections
- Road and cable network planning
- Clustering applications

## 🖨️ Concepts Covered

This project helps demonstrate the following concepts:

- Graph representation
- Graph traversal
- Queue
- Stack
- Recursion
- Weighted graphs
- Greedy algorithms
- Minimum Spanning Tree
- Cycle detection
- Union-Find / Disjoint Set
- Time and space complexity

## 🚀 Learning Objective

The main objective of this project is to understand how different graph algorithms work and how their performance differs depending on the graph structure.

By implementing these algorithms in Python with user input, students can practice both **algorithm design** and **complexity analysis**.

## 👨‍💻 Author

**Sricharan Yenugula**

GitHub: `https://github.com/your-username`

## 📄 License

This project is created for **educational and learning purposes**.

# Travelling Salesman Problem (TSP)

## 📌 Introduction

The Travelling Salesman Problem (TSP) is a classic optimization problem in computer science. The goal is to find the shortest possible route that visits every city exactly once and returns to the starting city.

This program is implemented in Python and accepts user input for the number of cities and the distance between them.

## 🎯 Objectives

- Understand the Travelling Salesman Problem.
- Find the minimum-cost tour between cities.
- Accept distances from the user.
- Calculate and display the minimum tour cost and route.

## ⚙ Algorithm

1. Start the program.
2. Read the number of cities from the user.
3. Read the distance matrix.
4. Generate possible tours starting from the first city.
5. Calculate the total distance of each tour.
6. Select the tour with the minimum total distance.
7. Display the best route and its cost.
8. Stop the program.

## 💻 Requirements

- Python 3.x
- Basic knowledge of Python and graph algorithms

## ▶ How to Run

1. Save the Python program as `tsp.py`.
2. Open a terminal in the project folder.
3. Run the following command:

```
        python tsp.py
```

4. Enter the number of cities and the distance matrix when prompted.

## 🕹 Sample Input

```
Enter number of cities: 4

Enter the distance matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

## 🕹 Sample Output

```
Minimum cost: 80
Optimal tour: 1 -> 2 -> 4 -> 3 -> 1
```

*Note: The output assumes a symmetric distance matrix and a program that searches all possible tours.*

## ⏱ Time and Space Complexity

- **Time Complexity:** O(n!)
- **Space Complexity:** O(n), excluding the distance matrix, for an implementation that evaluates tours without storing all permutations.

## 🌍 Applications

- Delivery route optimization
- Logistics and transportation
- Network design
- Circuit board manufacturing
- Planning travel routes

## 📁 Project Structure

```
Travelling-Salesman/
├── tsp.py
└── README.md
```

## 📝 Conclusion

The Travelling Salesman Problem demonstrates how optimization algorithms can be used to find the shortest route among multiple cities. A brute-force solution is suitable for small inputs but becomes expensive as the number of cities increases.

# Floyd–Warshall Algorithm

## 📌 Introduction

The Floyd–Warshall algorithm is a graph algorithm used to find the shortest paths between **all pairs of vertices** in a weighted graph.

It uses dynamic programming to improve shortest-path solutions by considering each vertex as an intermediate point. The algorithm supports negative edge weights but does not work correctly when a negative-weight cycle exists.

## 🎯 Objectives

- Understand the all-pairs shortest-path problem.
- Implement the Floyd–Warshall algorithm using Python.
- Accept a weighted adjacency matrix from the user.
- Display the shortest distances between every pair of vertices.

## ⚙ Algorithm

1. Start the program.
2. Read the number of vertices.
3. Read the weighted adjacency matrix.
4. Initialize the distance matrix using the given weights.
5. Consider each vertex as an intermediate vertex.
6. Update the distance between every pair of vertices using:

   `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`
7. Display the final shortest-distance matrix.
8. Check for negative cycles if required.
9. Stop the program.

## 💻 Requirements

- Python 3.x
- Basic knowledge of graphs and dynamic programming

## ▶ How to Run

1. Save the Python program as `floyd_warshall.py`.
2. Open a terminal in the project folder.
3. Run the following command:

```
python floyd_warshall.py
```

4. Enter the number of vertices and the adjacency matrix when prompted.

Use `INF` to represent the absence of a direct edge, if your program supports that input format.

## 🕹 Sample Input

```
Enter number of vertices: 4

Enter the adjacency matrix:
0 5 INF 10
INF 0 3 INF
INF INF 0 1
INF INF INF 0
```

## 🖨️ Sample Output

```
Shortest distance matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
```

## ⏱ Time and Space Complexity

- **Time Complexity:** $O(V^3)$
- **Space Complexity:** $O(V^2)$

Here, `V` represents the number of vertices in the graph.

## 🌍 Applications

- Network routing
- Road and transportation networks
- Finding shortest distances between all locations
- Graph analysis
- Communication networks

## 📁 Project Structure

```
Floyd-Warshall/
├── floyd_warshall.py
└── README.md
```

## 📝 Conclusion

The Floyd–Warshall algorithm efficiently calculates shortest-path distances between all pairs of vertices. It is useful for dense graphs and network problems where distances between many pairs of locations are needed.